PROJECT ROADMAP

# AI-Driven Integrated Business Management System

## for Smart Billing, Inventory Control, and Profit Prediction

*A 14-Week Execution Plan for a 5-Member Team*

Prepared by: **Debanjan Ghosal** — Team Lead

---

## 1. Project Overview

This roadmap is built specifically around your title — an integrated system where billing, inventory, and profit prediction are not three separate demos but one connected pipeline: every sale recorded at billing updates inventory in real time, and the same transaction stream feeds the profit-prediction model. That connectedness is what will make the project feel like a genuine "system" rather than three unrelated modules bolted together.

Unlike a generic "business management system," your title commits you to three specific, defensible pillars. Every design decision below is chosen to make those three pillars deep and well-integrated rather than spreading effort across five or six loosely connected AI features.

### The Three Pillars

- **1) Smart Billing** — a POS/billing interface that records every transaction (items, quantities, price, discounts, tax) and is the single source of truth the rest of the system reads from.
- **2) Inventory Control** — real-time stock tracking driven by billing transactions, with AI-based reorder-point prediction and low-stock alerts so the system anticipates shortages instead of just logging them.
- **3) Profit Prediction** — a machine learning layer that forecasts product- and period-level profit from historical price, cost, discount, and sales-volume data, surfaced on a BI dashboard for decision-making.

## 2. System Architecture

A clean, layered architecture is one of the first things examiners look at, and it keeps five people from stepping on each other's code.

**Frontend → Backend API (REST) → Database ↔ ML Service Layer → BI Dashboard**

- Presentation layer:
  - Billing/POS screen, Inventory screen, Admin dashboard, and Profit-prediction dashboard — all React views hitting the same backend.
- Application layer:
  - A REST API (FastAPI or Node/Express) handling auth, billing transactions, inventory CRUD, and serving ML predictions through dedicated endpoints (e.g. /predict/profit, /predict/reorder).
- Data layer:
  - A single relational database (PostgreSQL) that both the transactional app and the ML training pipeline read from — this is what makes the system "integrated" rather than three silos.
- Intelligence layer:

- Scheduled/triggered ML jobs (batch retraining nightly or on-demand) that write predictions back into the database, which the dashboard then simply displays — keep the dashboard dumb and the ML layer smart.

Draw this as a formal architecture diagram (boxes and arrows) in your report — it is one of the most commonly requested figures in a viva and takes under an hour to make in draw.io or Figma.

## 3. Recommended Tech Stack

| Layer | Recommendation | Why |
|---|---|---|
| Frontend | React + Tailwind CSS | Fast to build, huge component ecosystem, easy to demo responsively |
| Backend | FastAPI (Python) | Same language as your ML code — no context-switching, auto-generated API docs |
| Database | PostgreSQL | Relational integrity for transactions/stock; well-suited to billing data |
| Billing/POS logic | Backend service + invoice PDF generation (reportlab / pdfkit) | Realistic POS behaviour: line items, tax, discount, printable receipt |
| Inventory ML | scikit-learn (Random Forest) for reorder-point; simple statistical safety-stock formula as baseline | Random Forest consistently outperforms linear models for stock-level prediction in published studies |
| Profit Prediction | scikit-learn: Gradient Boosting / Random Forest, benchmarked against Linear Regression | Comparative studies on retail profit data found Gradient Boosting and Random Forest gave the best accuracy versus plain regression |
| Demand forecasting (feeds profit model) | Prophet or ARIMA for time-series; optional LSTM only if time allows | Prophet/ARIMA are far quicker to implement and tune than LSTM for a student timeline |
| Dashboard/BI | Recharts or Plotly inside React | KPI cards + trend charts read directly from your API |
| Auth | JWT-based auth, role-based access (Admin / Cashier / Manager) | Realistic for a business system; also gives you a natural “security” talking point |
| Deployment | Docker Compose locally; Render/Railway for a live demo link | Cheap, fast, avoids DevOps rabbit holes |

## 4. Database / Data Model

Design this as an ER diagram before writing a single migration. The Predictions_Log table is easy to forget but important: it lets you show predicted-vs-actual accuracy over time in your report, which is far more convincing than a single accuracy number.

| Entity | Key Fields | Feeds |
|---|---|---|
| Products | id, name, category, cost_price, selling_price, current_stock, reorder_level | Billing, Inventory, Profit model |

| | | |
|---|---|---|
| Transactions / Invoices | id, timestamp, cashier_id, customer_id, total, tax, discount, payment_mode | Billing, Profit model, Dashboard |
| Transaction_Items | transaction_id, product_id, quantity, unit_price, line_total | Inventory deduction, Profit model |
| Customers | id, name, contact, loyalty_points (optional) | Dashboard, optional recommendations |
| Suppliers | id, name, contact, lead_time_days | Reorder-point calculation |
| Stock_Movements | product_id, change_qty, reason (sale/restock/adjustment), timestamp | Inventory intelligence, audit trail |
| Users | id, name, role (admin/manager/cashier), password_hash | Auth, role-based views |
| Predictions_Log | product_id/period, predicted_profit, predicted_reorder_point, model_version, generated_at | Dashboard, model evaluation over time |

# 5. The AI/ML Layer — In Detail

## 5.1 Smart Billing Intelligence

Billing itself is largely transactional (CRUD + invoice generation), but you can legitimately make it "smart" without over-scoping:

- Auto-apply discount/tax rules based on product category or bulk quantity.
- Anomaly flagging: detect unusual transactions (e.g. abnormally high discount, refund spike) using a simple z-score or isolation-forest check — a small, demoable AI touch that examiners like.
- Barcode/QR-based item lookup speeds up billing and doubles as your inventory update trigger (a webcam + QR library is a well-documented approach in published smart-inventory prototypes).

## 5.2 Inventory Control Intelligence

This is where "control" should mean prediction, not just tracking:

- Reorder-point prediction: train a regression/Random Forest model on historical sales velocity, lead time, and seasonality to predict when each product will hit a critical stock level — more adaptive than a static "reorder at 10 units" rule.
- Low-stock alerts: rule-based trigger layered on top of the predicted reorder point, surfaced on the dashboard and optionally emailed to the manager.
- Demand forecasting: Prophet or ARIMA per product/category to project the next 7–30 days of demand; this output directly feeds the profit-prediction model below, which is what makes the three pillars genuinely connected rather than parallel features.

## 5.3 Profit Prediction — the centerpiece module

This is the most distinctive part of your title, so it deserves the most rigor. Published comparative studies on retail profit prediction consistently find that ensemble tree methods beat plain regression:

- Feature set: unit price, cost of goods sold, discount %, stock level, sales volume/velocity, category, and (optionally) a seasonality indicator from your demand-forecast output.
- Model comparison: train and report at least three models — Linear Regression (interpretable baseline), Random Forest, and Gradient Boosting (XGBoost/LightGBM) — and show the comparison in a table. Studies on similar retail

profit datasets found Gradient Boosting and Random Forest gave noticeably better accuracy than ordinary least squares, while Random Forest reached an $R^2$ around 0.89 versus roughly 0.68 for linear regression in one inventory-forecasting study — use that kind of gap as your expected story, not a guaranteed number.

- Granularity: predict at both the per-product level ("which items are most/least profitable") and the period level ("projected profit next week/month") — two outputs from one well-engineered pipeline.
- Present it as decision support, not a black box: pair each prediction with the top contributing features (e.g. via feature_importances_ or SHAP) so the dashboard can say "profit is projected to dip — largely driven by rising COGS on Category X," which is a much stronger viva answer than a single number.

### 5.4 Optional stretch add-ons (only if the 3 pillars finish early)

Your title does not require these — do not let them eat into core-module time. Only pursue these if your group is genuinely ahead of schedule after Week 11 (profit prediction complete and integrated); otherwise skip them entirely and use the extra time as buffer:

- A simple rule-based FAQ chatbot for billing/inventory queries ("what's the stock of X?") — keep it intent-based, not a full RAG pipeline.
- A lightweight product recommendation widget (cosine-similarity based) on the billing screen.

## 6. 14-Week Roadmap — Day-by-Day Breakdown

Two extra weeks have been added versus a standard 12-week plan, specifically to give you real buffer time: Weeks 12–14 are dedicated to integration testing, documentation, and viva rehearsal rather than being squeezed in alongside new development. Weeks 3–11 run two workstreams in parallel — the transactional app (billing/inventory) and the ML pipeline — so your ML-focused member can start on data work in Week 8 while the rest of the team is still deep in the app. Adjust owners and day splits to your team's actual pace; treat the day ranges as a target rhythm, not a rigid contract.

| Week | Focus | Days | Tasks | Owner(s) |
|---|---|---|---|---|
| Week 1 | **Planning & Scope Lock** | Day 1–2 | • Finalize the exact business domain (e.g. one supermarket/pharmacy) — shapes every dataset and screen decision after this<br>• Write 1 paragraph per pillar: what it does, what data it needs, what "done" looks like | **Whole team (led by you)** |
| | | Day 3–5 | • Draw ER diagram and system architecture diagram<br>• Finalize tech stack and confirm everyone can run it locally | |
| | | Day 6–7 | • Write requirements doc (functional + non-functional, 1–2 pages)<br>• Set up repo, branching strategy, and task board (Trello/Jira/GitHub Projects) | |
| Week 2 | **Design Finalization & Setup** | Day 1–2 | • Dev environment set up for all 5 members<br>• Agree on API contracts — request/response shapes for every planned endpoint | **Whole team** |
| | | Day 3–5 | • Implement full DB schema as migrations (Products, Transactions, Inventory, Users, etc.) | |

| Week | Focus | Days | Tasks | Owner(s) |
|---|---|---|---|---|
| | | Day 6–7 | • Wireframes/mockups for billing, inventory, and dashboard screens<br>• Confirm design system (colors, components) in Tailwind config | |
| Week 3 | Backend Foundations | Day 1–3 | • Auth implementation + role-based access (admin/manager/cashier) | Member 1 (Backend/Auth) + Member 2 (DB) |
| | | Day 4–7 | • Product & Supplier CRUD APIs<br>• Unit tests for auth and product endpoints | |
| Week 4 | Inventory Base Module | Day 1–3 | • Inventory CRUD APIs, stock fields wired to Products table | Member 2 (DB/Inventory) |
| | | Day 4–7 | • Stock movement logging table + API<br>• Low-stock threshold fields; test all endpoints via Postman/Swagger | |
| Week 5 | Billing Module — Part 1 | Day 1–4 | • Create-invoice flow: line items, quantity, price capture | Member 3 (Billing) |
| | | Day 5–7 | • Tax/discount rule logic<br>• Receipt (PDF) generation | |
| Week 6 | Billing–Inventory Integration (Milestone) | Day 1–3 | • Wire inventory deduction to billing transactions in real time | Member 3 + Member 2 |
| | | Day 4–5 | • Stock-movement audit trail tied to each transaction | |
| | | Day 6–7 | • End-to-end test: one product, one sale — stock updates correctly. This is your first working vertical slice. | |
| Week 7 | BI Dashboard Shell | Day 1–3 | • KPI cards: revenue, stock turnover, top products | Member 4 (Frontend/Dashboard) |
| | | Day 4–5 | • Charts: sales trend, inventory levels, category breakdown | |
| | | Day 6–7 | • Role-based dashboard views (admin vs manager vs cashier) | |
| Week 8 | Data Pipeline Foundation | Day 1–3 | • Source a real Kaggle retail dataset; design the synthetic-data augmentation plan | Member 5 (ML) + you |
| | | Day 4–5 | • Data cleaning + feature engineering pipeline shared by inventory and profit models | |
| | | Day 6–7 | • Train/test split strategy; baseline (linear) models running end-to-end | |
| Week 9 | Inventory Intelligence | Day 1–4 | • Reorder-point prediction model (Random Forest): training + tuning | Member 5 (ML) |
| | | Day 5–7 | • Low-stock alert logic<br>• Demand forecasting model (Prophet/ARIMA) | |

| Week | Focus | Days | Tasks | Owner(s) |
|---|---|---|---|---|
| | | | per product/category | |
| Week 10 | Profit Prediction Models | Day 1–4 | • Train and tune Linear Regression, Random Forest, and Gradient Boosting | Member 5 (ML) |
| | | Day 5–7 | • Build the model-comparison table (RMSE/MAE/$R^2$)<br>• Feature-importance / SHAP explainability output | |
| Week 11 | ML–Dashboard Integration | Day 1–3 | • Build /predict/profit and /predict/reorder API endpoints | Member 5 + Member 4 |
| | | Day 4–5 | • Connect predictions to dashboard: profit forecast cards, reorder alerts | |
| | | Day 6–7 | • Predictions_Log wired up; RMSE/MAE/$R^2$ displayed on an evaluation panel | |
| Week 12 | Full Integration Testing | Day 1–3 | • End-to-end integration testing: billing → inventory → prediction → dashboard | Whole team |
| | | Day 4–5 | • Bug bash and fixes across all modules | |
| | | Day 6–7 | • UI polish, performance/load check, edge-case testing | |
| Week 13 | Documentation & Report | Day 1–3 | • Architecture, schema, and model documentation | Whole team (led by you) |
| | | Day 4–5 | • Full project report: problem statement, evaluation results, limitations | |
| | | Day 6–7 | • Slide deck creation; record/script the demo | |
| Week 14 | Viva Prep, Buffer & Submission | Day 1–2 | • Individual module Q&A prep — each member owns their module | Whole team |
| | | Day 3–4 | • Full mock viva / dry-run presentation to the group or a mentor | |
| | | Day 5–6 | • Buffer for last-minute fixes based on mock-viva feedback | |
| | | Day 7 | • Final submission | |

## 7. Suggested Team Role Allocation

With five people, avoid a “everyone does everything” split — it causes integration chaos late in the project. One clear owner per pillar, with the whole team converging for integration weeks (1–2, 12–14), works well in practice.

| Role | Primary Responsibility | Also Owns |
|---|---|---|
| Team Lead — Debanjan Ghosal | Architecture, integration, scope control, report/viva coordination | Requirements doc, final documentation |
| Member 1 | Backend & Auth | REST API structure, role-based access, |

| | | |
|---|---|---|
| | | deployment/Docker |
| Member 2 | Database & Inventory | Schema, migrations, stock-movement logic, inventory endpoints |
| Member 3 | Billing/POS | Invoice flow, tax/discount rules, receipt generation |
| Member 4 | Frontend & Dashboard | React UI, charts/KPI cards, role-based views |
| Member 5 | ML/Data Science | Reorder prediction, demand forecasting, profit-prediction models, evaluation |

### 7.1 Weekly Cadence & Escalation

A short, regular sync keeps five people moving as one team instead of five separate projects that happen to share a repo. Suggested rhythm, run by Debanjan Ghosal as Team Lead:

- Weekly standup (15–20 min, fixed day/time): each member gives what was done, what's next, and any blocker — no problem-solving in the standup itself.
- Blockers escalate to the Team Lead the same day, not at the next standup — a blocked ML or integration task can quietly cost 2–3 days if it waits a week to surface.
- End-of-week check-in: Team Lead confirms each week's deliverable (from Section 6) actually shipped before the next week's tasks start — this is what keeps the 14-week schedule honest.
- Milestone reviews at the end of Weeks 2, 6, 11, and 14: whole-team demo of what's working, aligned to the Section 6 milestones (scope lock → billing–inventory slice → full ML integration → submission).

## 8. Evaluation Metrics

- Profit prediction & demand forecasting (regression):
  - RMSE, MAE, and $R^2$ — report all three per model, plus the model-comparison table (Linear vs Random Forest vs Gradient Boosting).
- Reorder-point / low-stock alerts (classification-flavoured):
  - Precision/Recall/F1 if you frame “will stock out in N days” as a classification problem; otherwise RMSE on predicted days-to-stockout.
- System-level metrics:
  - Billing transaction throughput/latency, inventory-update consistency (no phantom stock), dashboard load time.
- Business-facing metrics for your report narrative:
  - Reduction in simulated stockout events, % of profit-driving products correctly identified in top-N — easy to compute on your own synthetic/test data and much more persuasive in a viva than raw accuracy alone.

### 8.1 Success Criteria (Definition of “Done”)

Vague goals like “the model should work well” are hard to defend in a viva. Agree on concrete pass/fail thresholds up front:

- Every ML model (profit prediction, reorder-point, demand forecast) must beat a naive baseline — e.g. Random Forest/Gradient Boosting must outperform plain Linear Regression, and the forecast must beat “predict last period's value” — on the same held-out test set.

- The full pipeline — one sale entered at billing → stock decrements → dashboard updates → a prediction refreshes — must be demonstrable live, end-to-end, without manual database edits.
- Each of the three pillars has at least one working screen and one trained model by the end of Week 11, independent of how polished the UI is — polish is a Week 12–14 concern, not a Week 11 one.
- Every reported metric (RMSE/MAE/$R^2$/precision/recall) is reproducible from a documented script or notebook — no numbers in the report that can't be regenerated on demand if an examiner asks.

## 9. Risks & Mitigations

| Risk | Mitigation |
|---|---|
| Scope creep across 3 AI-heavy pillars | Lock scope in week 1–2 using the "1 paragraph per module" exercise; stretch add-ons are explicitly optional and last-priority |
| Cold-start / unrealistic profit numbers on synthetic data | Blend a real Kaggle retail dataset (e.g. Online Retail, Superstore Sales) with synthetic data engineered for seasonality/margin patterns |
| Integration between billing → inventory → ML → dashboard breaks late | Build a thin end-to-end slice through billing → inventory by Week 6, then extend it through prediction → dashboard by Week 11, well before the Week 12–14 testing/documentation buffer |
| ML member becomes a bottleneck | Have the ML owner start dataset research and light exploration from Week 3 onward, even though the full pipeline build begins in Week 8 — don't let data sourcing become a Week 8 surprise |
| Overselling model accuracy | Report metrics honestly with dataset caveats (real vs. synthetic); examiners respect honest limitations sections |
| Five people, five coding styles | Agree on API contracts (request/response shapes) in week 1–2 so modules can be built independently and integrated without rewrites |
| Team member unavailable (illness, exams, personal emergency) | No module has only one person who understands it — pair members on the two riskiest modules (Billing–Inventory integration, Profit Prediction) so work can continue if someone is out |
| Live demo/viva-day technical failure (deployment down, wifi, laptop issues) | Always have a local copy running as backup, plus a pre-recorded demo video ready to play instantly if the live version fails |
| Project timeline collides with exams or other coursework | Treat Weeks 9–11 (core ML work) as the least flexible weeks — if a clash is unavoidable, absorb it in Weeks 12–14 buffer instead, not by cutting ML scope |
| Billing module handles customer data | Use anonymized/synthetic customer records for the demo — no real names, phone numbers, or payment details — and mention this as a stated non-functional requirement in your report |

## 10. Final Deliverables Checklist

- Working end-to-end system: billing → real-time inventory update → profit/reorder predictions → dashboard
- Source code repository with README (setup instructions, architecture diagram, schema)
- Project report: problem statement, architecture, data model, model comparison tables, evaluation results, limitations
- Presentation deck with a live demo, plus a pre-recorded backup video in case the live demo fails on the day
- Model documentation: features used, algorithms compared, final chosen model and why

- Individual viva prep: each member should be able to explain their module end-to-end, including why alternative approaches were not chosen

### 10.1 Suggested Report Chapter Outline

A ready-made structure for the “Project report” deliverable above — fill each chapter in as the corresponding weeks complete rather than writing it all in Week 13:

- 1. Abstract — one paragraph: problem, approach, key result
- 2. Introduction — problem statement, objectives, scope (your three pillars)
- 3. Literature Review / Related Work — how existing smart-billing, inventory, and profit-prediction systems approach this, and where yours differs
- 4. System Design — architecture diagram, ER diagram, tech stack and why (Sections 2–4 of this roadmap map directly here)
- 5. Implementation — module-by-module, one subsection per pillar, one contributor named per subsection
- 6. AI/ML Methodology — features, models compared, why the final model was chosen (Section 5 of this roadmap maps here)
- 7. Results & Evaluation — metrics tables, predicted-vs-actual charts, honest discussion of limitations (Sections 8–9 map here)
- 8. Conclusion & Future Work — what worked, what you'd improve, and the optional stretch add-ons as “future work” if not built
- 9. References — every paper, dataset, or library cited

## Final Note

*The strongest version of this project is not the one with the most features — it's the one where a marker can trace a single sale from the billing screen, watch stock decrement, and see it reflected in a profit forecast a moment later. Build that thin, fully-connected slice through billing → inventory by Week 6, extend it through prediction → dashboard by Week 11, and leave Weeks 12–14 purely for testing, documentation, and viva prep — don't let new development bleed into those final weeks. That sequencing is the single biggest driver of a smooth final month.*